基于多样性的深度强化学习实现格斗游戏AI的多维难度

Emily Halina, Matthew Guzdial

加拿大阿尔伯塔省埃德蒙顿市阿尔伯塔大学计算科学系、阿尔伯塔机器智能研究所（Amii）{ehalina, guzdial}@ualberta.ca

摘要

在格斗游戏中，相同技能水平的个体玩家之间往往会在游戏过程中表现出截然不同的策略。然而，大多数格斗游戏AI智能体在每个“难度等级”上仅采用单一策略。为了让AI对手更像人类，我们理想中希望在每个难度等级上都能看到多种不同的策略，我们将这一概念称为“多维”难度。本文提出了一种基于多样性的深度强化学习方法，用于生成一组难度相近但策略多样的AI智能体。我们发现，这种方法在多样性和性能方面均优于使用专门的人工编写的奖励函数训练的基线模型。

引言

格斗游戏长期以来一直设有作为人类玩家对手的AI智能体。这些AI智能体大多基于“线性”难度概念创建，这意味着不同智能体之间的唯一区别在于一个从简单到困难的固定难度评级。尽管这种线性难度模型很普遍，但它与普通玩家与其他人类对战的体验并不完全契合。格斗游戏中丰富的玩家表现使得技能水平相近的人类玩家能够以不同方式运用相同机制，给对手带来独特的挑战（Dhami 2021）。例如，一名玩家可能利用角色的技能保持距离并逐渐消耗对手，而另一名玩家则可能使用相同的技能在近距离内激进地逼近对手。这种脱节会让玩家感觉在面对其他人类对手时准备不足，甚至可能导致部分玩家完全放弃某款游戏。

这一问题可通过引入“多维难度”系统加以缓解，在该系统中，智能体不仅通过线性难度加以区分，还具备诸如游戏风格等额外特质。据我们所知，这是首个识别该研究问题的学术成果。通过融入多维难度的概念，格斗游戏能够为玩家提供与多种多样策略对战的体验。这能让玩家更好地为与其他人类玩家对战做好准备，并促使他们更充分地探索游戏机制。

实现多维度难度的主要挑战在于创建格斗游戏AI智能体的整体设计负担。格斗游戏智能体通常需要大量的手动编写才能发挥效果，使用基于规则的系统或有限状态机来适应复杂的游戏机制和角色交互（Majchrzak、Quadflieg 和 Rudolph 2015）。由于这一困难，一些游戏会通过读取玩家输入来“作弊”，人为增加AI智能体的难度，进一步破坏了人类与AI对手之间的公平性。此前已有研究通过使用强化学习技术来减轻这一设计负担，自主训练AI智能体玩格斗游戏（Kim、Park 和 Yang 2020；Oh 等人 2021）。然而，这些工作大多仅专注于玩游戏本身，这足以说明开发这类智能体的难度。因此，自动开发具有多样化策略的格斗游戏智能体这一问题相对而言尚未得到充分探索。

在本文中，我们专注于训练一组难度相近且彼此采用多样化策略的智能体的任务。理想情况下，这些智能体将为玩家提供更全面、稳健的游戏体验，同时带来适当的挑战。

为实现这一目标，我们提出了Brisket——一种受(Eysenbach等人 2018)的“多样性即是一切”(DIAYN)方法启发、基于多样性的深度强化学习方法，用于学习一套技能水平相近但策略多样化的行为。我们在FightingICE中实现了Brisket，这是一个为测试和评估AI智能体而构建的格斗游戏研究平台(Lu等人 2013)。我们评估了Brisket学到的策略，将其与一组使用专门的人工编写的奖励函数训练的“人工编写”基线智能体进行对比，发现Brisket在有效性和多样性两方面均优于这些基线智能体。因此，我们主张基于多样性的强化学习方法可以成为为具有多维难度的格斗游戏生成敌方AI智能体的有效途径。

arXiv:2211.02759v1 [cs.LG] 4 Nov 2022

Diversity-based Deep Reinforcement Learning Towards Multidimensional Difficulty for Fighting Game AI

Emily Halina, Matthew Guzdial

Department of Computing Science, Alberta Machine Intelligence Institute (Amii)
University of Alberta, Edmonton, Alberta, Canada
{ehalina, guzdial}@ualberta.ca

Abstract

In fighting games, individual players of the same skill level often exhibit distinct strategies from one another through their gameplay. Despite this, the majority of AI agents for fighting games have only a single strategy for each “level” of difficulty. To make AI opponents more human-like, we’d ideally like to see multiple different strategies at each level of difficulty, a concept we refer to as “multidimensional” difficulty. In this paper, we introduce a diversity-based deep reinforcement learning approach for generating a set of agents of similar difficulty that utilize diverse strategies. We find this approach outperforms a baseline trained with specialized, human-authored reward functions in both diversity and performance.

Introduction

Fighting games have long featured AI agents that act as opponents for human players to play against. The majority of these AI agents are created using a notion of “linear” difficulty, meaning the only distinction between agents is a difficulty rating in a fixed range from easy to hard. Despite its prevalence, this model of linear difficulty is not well aligned with the average player’s experience playing against other humans. The high amount of player expression within fighting games allows for human players of similar skill levels to use the same mechanics in disparate ways to uniquely challenge their opponents (Dhami 2021). For example, one player may use a character’s tools to keep their distance and chip away at an opponent, while another may use the same tools to aggressively approach the opponent in close quarters. This disconnect can make players feel inadequately prepared for playing against other humans, and could cause some to drop a game entirely.

This problem could be mitigated through the use of a “multidimensional” difficulty system in which agents are distinguished by both linear difficulty and additional qualities such as playstyle. To the best of our knowledge, this is the first academic work to identify this research problem. By incorporating a notion of multidimensional difficulty, fighting games could provide players the experience of playing against multiple diverse strategies. This could allow players to better prepare for playing against other human players and push them to more fully explore the mechanics of the game.

A major challenge in attaining multidimensional difficulty is the overall design burden of creating fighting game AI agents. Fighting games agents often require significant hand-authoring to be effective, using rules-based systems or Finite State Machines (FSMs) to accommodate complex game mechanics and character interactions (Majchrzak, Quadflieg, and Rudolph 2015). Due to this difficulty, some games resort to “cheating” by reading the player’s inputs to artificially increase the difficulty of an AI agent, further breaking parity between human and AI opponents. There has been prior work in alleviating this designer burden through the use of Reinforcement Learning (RL) techniques to autonomously train AI agents to play fighting games (Kim, Park, and Yang 2020; Oh et al. 2021). However, the majority of this work has been focused solely on playing the game, which is a testament to the difficulty of developing such agents. As such, the problem of automatically developing agents that exhibit diverse strategies for fighting games has been relatively unexplored.

In this paper we focus on the task of training a group of agents of similar difficulty that utilize diverse strategies from one another. Ideally, these agents would provide a more complete, robust gameplay experience for players while providing a suitable challenge.

Towards this goal, we propose Brisket, a diversity-based deep RL approach for learning a set of equally skilled, diverse strategies inspired by (Eysenbach et al. 2018)’s Diversity is All You Need (DIAYN). We implemented Brisket in FightingICE, a fighting game research platform built for the testing and evaluation of AI agents (Lu et al. 2013). We evaluated the policies learned by Brisket against a set of “human-authored” baseline agents trained with specialized, human-authored reward functions, and found that they outperformed these baseline agents in both effectiveness and diversity. Therefore, we claim that a diversity-based RL approach can be an effective way to produce enemy AI agents for fighting games with multidimensional difficulty.

相关工作

非玩家角色生成

针对许多游戏领域，已开发出通过算法生成非玩家角色行为与机制的方法。这些方法介于自动游戏玩法和程序化内容生成(PCG)之间，后者被定义为通过人工智能或算法手段生成游戏内容(Hendrikx等人 2013)。先前的相关工作中，非玩家角色生成任务常通过生成完整的游戏或游戏机制来涵盖(Guzdial和Riedl 2018; Cook, Colton和Gow 2016)。而我们将重点放在使用预定义机制集生成非玩家角色行为上。

先前的非玩家角色行为生成系统主要采用了构造性、基于搜索和基于约束的程序化内容生成技术（Barros, Liapis, and Togelius 2016; Mitsis et al. 2020）。其中，由（Butler, Siu, and Zook 2017）提出的一个系统利用程序合成为具有预定义约束的独特Boss行为生成。这些现有方法并不专注于生成多样化的非玩家角色群体，而是侧重于单个非玩家角色的生成。质量多样性搜索算法可用于生成此类群体，并已被应用于关卡生成和3D模型创建（Gravina et al. 2019）。尽管据我们所知，质量多样性搜索尚未应用于非玩家角色生成，但这可能成为解决我们问题的一种替代方法。

格斗游戏AI

在创建用于格斗游戏的AI智能体方面已有大量先前工作。部分前期工作侧重于利用手工编写的基于规则的系统来创建具有高度设计可控性的竞争性智能体（Sato et al. 2015）。（Majchrzak, Quadflieg, and Rudolph 2015）引入了这样一个基于规则的系统，它通过从人工编写的规则库中选择规则，利用动态脚本技术使AI适应特定对手。基于规则的系统可以被专门编写以实现期望的定性行为，例如（Yuda, Mozgovoy, and Danielewicz-Betz 2019）用于创建基于情感评估引擎做出决策的“情感”智能体的系统。基于规则系统的一个弱点是需要专家设计师进行大量手工编写工作。相比之下，我们的方法侧重于在不直接人工编写智能体行为的情况下学习策略。

近年来，基于蒙特卡洛树搜索（MCTS）和强化学习的格斗游戏智能体变得越来越占主导地位，其中一些智能体甚至击败了顶级人类玩家（Kim和Ahn 2018；Oh等人2021; Kim, Park和Yang 2020）。这些研究大多侧重于尽可能完美地玩游戏，而非生成新颖或多样化的策略。一个显著的例外是（Ishii等人2018）的系统，它将人工编写的非玩家角色“角色性格”整合到MCTS智能体的决策过程中。虽然该系统使用相同的架构创建了采用不同策略的智能体，但这些策略需要根据其偏好的行动进行手动编写。下文描述的人工编写的基线，正是从这项工作中汲取了灵感。

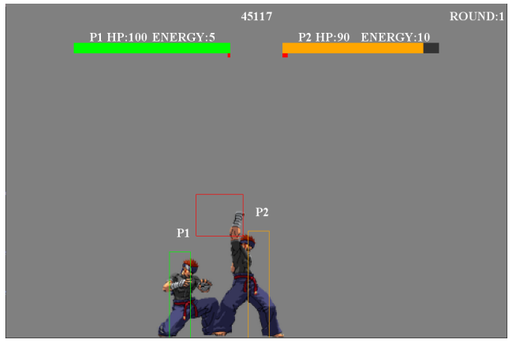

图1：FightingICE平台截图。每个玩家的生命值和能量资源显示在屏幕顶部。当AI智能体执行动作时，会生成一个临时的红色碰撞框。若对手的受击框与此碰撞框相交，则会受到伤害。

PCGRL

通过强化学习进行程序化内容生成（PCGRL）是指利用强化学习技术生成游戏内容（Khalifa等人2020）。大多数先前的PCGRL工作集中于关卡生成和谜题设计（Zakaria, Fayek和Hadhoud 2022；Delarosa等人2021）。我们的方法Brisket，其灵感来源于DIAYN，而DIAYN又受到先前基于多样性的强化学习和进化方法的启发（Eysenbach等人2018；Gregor, Rezende和Wierstra 2016；Such等人2017）。据我们所知，将基于多样性的强化学习应用于非玩家角色生成领域是全新的尝试。虽然Brisket受到DIAYN的启发，但我们对算法进行了修改，以更好地适应我们的领域和问题。

FightingICE

本节我们将简要介绍FightingICE——我们作为Brisket方法测试平台的环境。FightingICE是一个为格斗游戏AI智能体的开发与评估而构建的开源研究平台（Lu等，2013）。我们选择使用FightingICE，是因为该平台对人工智能开发的支持、其相对于其他格斗游戏较为简单的特性，以及其作为研究平台已被验证的成功经验（Chen等，2021；Ishihara等，2016）。

图1展示了FightingICE中两个AI智能体对战的屏幕截图。FightingICE是一款简单的二维格斗游戏，角色可使用多种拳击、踢击和基础闪避动作。角色还可使用消耗能量的特殊招式，能量是通过击中对手或被对手击中而获得的有限资源。FightingICE目前包含三个角色，但为简化起见，我们依照先前研究（Chen等，2021），将焦点限定在默认角色ZEN上。Brisket是基于该平台的4.50版本开发并测试的。

值得注意的是，由于FightingICE是为比较

Related Work

NPC Generation

Methods of algorithmically generating NPC behaviours and mechanics have been developed for many game domains. These methods fall between automated game playing and Procedural Content Generation (PCG), defined as the generation of game content through AI or algorithmic means (Hendrikx et al. 2013). The task of NPC generation is often included in prior work through the generation of entire games or game mechanics (Guzdial and Riedl 2018; Cook, Colton, and Gow 2016). We instead focus on the generation of NPC behaviour with a predefined set of mechanics.

Prior systems to generate NPC behaviour generation have used constructive, search-based, and constraint-based PCG techniques (Barros, Liapis, and Togelius 2016; Mitsis et al. 2020). One such system introduced by (Butler, Siu, and Zook 2017) utilized program synthesis for the generation of unique boss behaviour with pre-defined constraints. These existing methods do not focus on generating a diverse population of NPCs, but rather the generation of individual NPCs. Quality-diversity search algorithms exist for the generation of such populations, and have been applied to level generation and 3D model creation (Gravina et al. 2019). While to the best of our knowledge quality-diversity search has not been applied to NPC generation, this could represent an alternative approach to our problem.

Fighting Game AI

There has been a vast amount of prior work on the creation of AI agents for playing fighting games. Some prior work focuses on utilizing hand-authored rules-based systems to create competitive agents with a high amount of designer controllability (Sato et al. 2015). (Majchrzak, Quadflieg, and Rudolph 2015) introduced one such rules-based system that utilized dynamic scripting to adapt an AI to a given opponent by selecting rules from a human-authored corpus. Rules-based systems can be specifically authored to achieve desired qualitative behaviour, such as in (Yuda, Mozgovoy, and Danielewicz-Betz 2019)'s system for creating an “affective” agent that makes decisions based on an emotional appraisal engine. One weakness of rules-based systems is the intensive amount of hand-authoring required by expert designers. In comparison, our method focuses on learning policies without direct human authoring of an agent's behaviour.

In recent years, Monte Carlo Tree Search (MCTS) and RL-based fighting game agents have become more dominant, with some agents defeating top rated human players (Kim and Ahn 2018; Oh et al. 2021; Kim, Park, and Yang 2020). The majority of these works focus on playing the game as well as possible as opposed to generating novel or diverse strategies. A notable exception is (Ishii et al. 2018)'s system for incorporating human-authored NPC “personas” into an MCTS agent's decision making. While this system created agents using the same architecture that utilized distinct strategies, these strategies required authoring in terms of their preferred actions. Our human-authored baseline, described below, takes inspiration from this work.

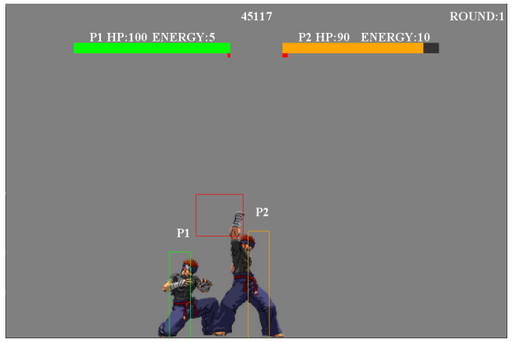

Figure 1: A screenshot of the FightingICE platform. Health and energy resources for each player are displayed at the top of the screen. When an agent performs a move, a temporary red hitbox is created. This hitbox will damage the opponent if their hurtbox intersects with it.

PCGRL

Procudural Content Generation via Reinforcement Learning (PCGRL) refers to the generation of game content via the use of RL techniques (Khalifa et al. 2020). Most prior PCGRL work focuses on level generation and puzzle design (Zakaria, Fayek, and Hadhoud 2022; Delarosa et al. 2021). Our approach, Brisket, takes inspiration from DIAYN, which in turn was inspired by prior diversity-based RL and evolutionary methods (Eysenbach et al. 2018; Gregor, Rezende, and Wierstra 2016; Such et al. 2017). The application of diversity-based reinforcement learning to the domain of NPC generation is novel to the best of our knowledge. While Brisket is inspired by DIAYN, we modified the algorithm in order to better suit our domain and problem.

FightingICE

In this section we briefly introduce FightingICE, the environment we used as a testbed for our approach Brisket. FightingICE is an open-source research platform built for the development and evaluation of AI agents for fighting games (Lu et al. 2013). We chose to use FightingICE due to the platform's support of AI development, its relative simplicity in comparison to other fighting games, and its proven success as a research platform (Chen et al. 2021; Ishihara et al. 2016)

Figure 1 depicts a screenshot of a match being played between two AI agents in FightingICE. FightingICE is a simple 2D fighting game, where characters have access to a variety of punches, kicks, and basic evasive maneuvers. Characters also have access to special moves which consume energy, a limited resource gained from hitting the opponent or being hit. FightingICE currently contains three characters, but for simplicity we restrict our focus to the default character ZEN, as in prior work (Chen et al. 2021). Brisket was developed and tested on version 4.50 of the platform.

Notably, because FightingICE was designed for the com-

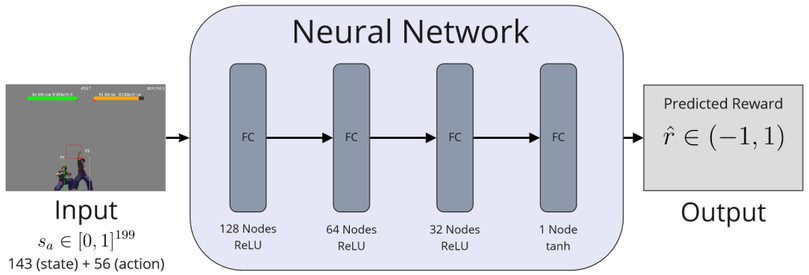

图2：DQN智能体架构的可视化。当前状态 s 和提议的动作 a 被拼接作为输入，通过一个全连接神经网络，该网络预测在状态 s 中执行动作 a 的预期奖励ˆr 。

自动游戏玩法的不同方法而设计的，因此将其用于NPC行为研究并非易事。为了让平台能够适配Brisket，必须进行多项调整，其中最显著的是移除了15帧的输入延迟。我们在GitHub仓库中提供了调整后的源代码，以及在FightingICE中实现的Brisket[1]。

系统概览

在本节中，我们讨论Brisket（我们基于多样性的深度强化学习方法）的技术细节。从高层来看，该方法涉及同时训练多个智能体，根据判别器，基于每个智能体与其他智能体的多样性给予奖励。然后，我们根据通用奖励函数对每个智能体单独进行微调，目标是创建难度相近但彼此多样的智能体。

我们首先讨论任务中马尔可夫决策过程的公式，描述状态和动作空间以及奖励函数。然后，我们详细介绍深度Q智能体和判别器所使用的架构。基于多样性的学习子节涵盖了训练方法中的多样性步骤，概述了用于“无奖励函数”训练智能体的算法，而微调子节则涵盖了训练方法中的微调步骤。

马尔可夫决策过程

一个MDP由状态空间 S、动作空间 A和奖励函数 R定义。我们忽略转移函数，因为我们的动作是确定性的。强化学习方法的目标是学习一个策略函数 π_R： $S \rightarrow A$，它将给定状态映射到该状态下的最优动作，以便根据 R最大化价值。

FightingICE 中没有默认的状态表示，因此我们必须自行设计。我们的状态表示包含 143 个变量，所有变量均为介于 0 和 1 之间的归一化浮点数。前 14 个变量包含关于两名玩家的基本信息，包括当前生命值、当前特殊招式的能量、位置和速度。接下来的 112 个变量表示每个玩家的当前状态，其中包含站立、蹲伏或执行动作等选项。该状态以独热编码向量表示，每个状态都与一个独特的动作相关联。我们将轮次中的剩余时间作为一个单一变量纳入，以支持更细致的决策，例如智能体在轮次末尾变得更加 Aggressive 或防守。最后 12 个变量编码了双方最近两个抛射物的相对位置和命中伤害。抛射物会在状态之间持续存在，直到击中玩家或到达其射程末端才停止移动。我们选择只包含最近的两个抛射物，以减小每个状态的大小，因为在任意给定时间同时存在更多抛射物的情况非常罕见。当没有抛射物或只有一个抛射物时，相应的未使用变量将被赋予占位值 0。

FightingICE的默认动作表示非常适合这项任务，因此我们只需枚举MDP动作空间中的所有可能动作即可。智能体可以执行56种可能的动作，包括简单选项（如向前行走或出拳）以及复杂输入（如抛射物或特殊招式）。这些动作被编码为长度为56的独热向量。

Brisket使用两个独立的奖励函数：微调函数 R_f 和基于多样性的函数 R_d。 R_d的定义和讨论如下。 R_f 定义为

$$R_f(s, a) = T(s) \cdot 1000 \tag{1}$$

其中 $T(s)$ 如果 s 为正终止状态（赢得游戏）， $T(s)$ 如果 s 为负终止状态（输掉游戏），否则为 $T(s)$。我们选择不引入存在性惩罚，以免使可能通过时间耗尽获胜的策略失效，因为状态表示中包含了轮次内的剩余时间。用于开发基线DQN智能体的额外专用奖励函数在评估部分中定义。

架构

我们的方法Brisket涉及同时训练固定数量的深度Q网络(DQN)智能体。DQN智能体利用神经网络来近似一个查找表，该表将给定的状态动作对映射到在该状态下采取该动作的期望价值（Li 2017）。在本小节中，我们将重点介绍每个DQN智能体所使用的架构，以及判别器所使用的架构，该判别器在基于多样性的训练步骤中确定奖励。

[1]https://github.com/emily-halina/brisket

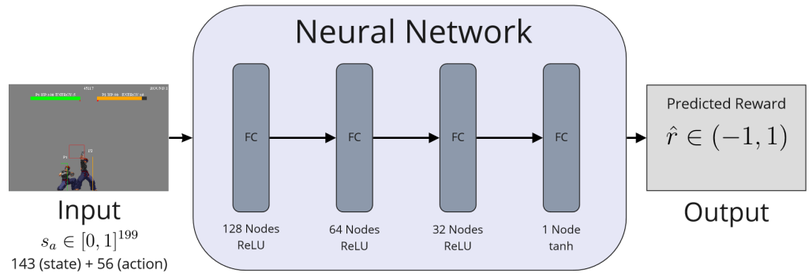

Figure 2: A visualization of the DQN agent architecture. The current state s and proposed action a are concatenated as input and passed through a fully connected neural network, which predicts the expected reward $\hat{r}$ of taking action a in state s.

parison of automated game playing approaches, it was not a trivial task to use it for NPC behaviour research. Multiple adjustments had to be made to the platform in order for it to function for Brisket, the most notable of which is the removal of a 15 frame input delay. We provide the adjusted source code in a GitHub repository, along with the implementation of Brisket in FightingICE[1].

System Overview

In this section we discuss the technical details of Brisket, our diversity-based deep RL approach. On a high level, the approach involves training multiple agents concurrently, rewarding each agent based on their diversity from one another according to a discriminator. We then fine-tune these agents individually on a general reward function with the goal of creating agents of a similar difficulty that are diverse from one another.

We begin by discussing the formulation of the Markov Decision Process (MDP) for our task, describing the state and action spaces and reward function. We then detail the architecture used by our deep Q-agents and discriminator. The Diversity-Based Learning subsection covers the diversity step of our training approach, outlining the algorithm used for training our agents “without a reward function,” and the Fine-tuning subsection covers the fine-tuning step of our training approach.

Markov Decision Process

An MDP is defined as a state space S, action space A, and reward function R. We ignore the transition function as we have deterministic actions. The goal of an RL approach is to learn a policy function $\pi_R : S \rightarrow A$ which maps a given state to the optimal action to take in that state to maximize value according to R.

There is no default state representation in FightingICE, and so we had to design our own. Our state representation contains 143 variables, all of which are floats normalized between 0 and 1. The first 14 variables contain basic information about both players, including current health, current energy for special moves, position, and velocity. The following 112 variables represent the current status of each player, which includes options such as standing, crouching, or performing a move. This status is encoded as a one-hot vector, with each status being associated with a unique action. We include the remaining time in the round as a single variable to allow for more nuanced decision making, such as an agent becoming more aggressive or defensive at the end of the round. The final 12 variables encode the relative position and hit damage of the two most recent projectiles from both players. Projectiles persist between states, continuing to move until colliding with a player or reaching the end of their range. We chose to include only the most recent two projectiles in an effort to reduce the size of each state, as it is very rare for more to be active at any given time. When there are no projectiles or only one projectile, the corresponding unused variables are given a placeholder value of 0.

FightingICE’s default action representation was a good fit for this task, and so we can just enumerate all possible actions for the action space of the MDP. There are 56 possible actions agents can take, including simple options such as walking forward or punching, along with complex inputs such as projectiles or special moves. These actions were encoded as an one-hot vector of length 56.

Brisket uses two separate reward functions: the fine-tuning function R_f and the diversity-based function R_d. R_d is defined and discussed below. R_f is defined as

$$R_f(s, a) = T(s) \cdot 1000 \tag{1}$$

where $T(s) = 1$ if s is a positive terminal state (winning the game), $T(s) = -1$ if s is a negative terminal state (losing the game), and $T(s) = 0$ otherwise. We chose not to include an existential penalty as not to invalidate strategies that may attempt to win by time-out, as the state representation includes the remaining time in a round. Additional specialized reward functions used for the development of our baseline DQN agents are defined in the Evaluation section.

Architecture

Our approach Brisket involves the training of a fixed number of Deep Q-Network (DQN) agents in tandem. DQN agents utilize a neural network to approximate a lookup table mapping a given state-action pair to the expected value of taking

[1]https://github.com/emily-halina/brisket

算法1：多样性学习算法

令 S_π 为固定策略集， D 为判别器
对于回合 e **执行**
  对于 轮次 r **执行**
    采样 $\pi_r \sim S_\pi$ 以在 r过程中做出选择
    对于时间步 t **执行**
      $a_t \leftarrow \epsilon - greedy(\pi_r, \ s_t)$
      对于策略 $\pi \in S_\pi$ **执行**
        // 基于 D计算奖励
        $r_{\pi,\ t} \leftarrow R_d\ (s_t,\ a_t,\ \pi,\ D)$
      结束循环
      步进环境 $s_{t+1} \leftarrow (s_t, a_t)$
    结束循环
  结束循环
  使用来自 e的样本更新每个策略 $\pi \in S_\pi$
  使用来自 e的样本更新判别器 D
结束循环

在本小节中，我们将重点介绍每个DQN智能体所使用的架构，以及判别器所使用的架构，该判别器在基于多样性的训练步骤中确定奖励。

图2展示了DQN架构的可视化。该模型接收一个长度为143的向量作为状态s，以及一个长度为56的独热向量作为待评估的提议动作 a 。这些向量被拼接后，依次通过3个分别由128、64和32个神经元组成的全连接层。每个全连接层均使用线性整流单元（ReLU）作为激活函数。我们的最终输出层是一个单节点，采用tanh激活函数来表示预测奖励 $\hat{r} \in (-1,1)$。ReLU的使用及模型参数设置参考了此前在fightingICE领域针对DQN智能体的研究，但一个显著变化是：模型不再被视为一个56类分类问题（Takano等人，2018年），而是基于状态动作对来预测奖励。这一变化旨在降低模型表征的复杂度，使模型能够更轻松地学习状态与各动作之间的关系。

Brisket在多样性步骤中使用一个判别器，以根据智能体的多样性给予奖励。该判别器以状态动作对作为输入，并预测该对来自每个智能体的可能性。这一可能性随后被用于这些智能体的奖励函数，鼓励它们彼此采取越来越多样化的动作。判别器采用与每个DQN智能体相同的架构，唯一的架构变化在于输出层。输出层有 K个节点，其中 K 是要学习的策略数量，并使用softmax激活函数代替tanh。我们选择为判别器和DQN智能体使用相同的架构，因为它们都在建模状态动作对与期望输出之间的相似关系。

基于多样性的学习

Brisket在很大程度上受到DIAYN的启发，DIAYN是一种用于同时训练一组强化学习智能体的方法，这些智能体根据判别器获得奖励，该判别器用于判断这些智能体之间的差异性程度（Eysenbach等人，2018年）。Brisket做出了一项重大改变，即为我们的DQN智能体和判别器使用状态-动作输入，而不仅仅是状态（Eysenbach等人，2018年）。这一改变是由于我们的行为生成任务与DIAYN最初设计的探索任务有所不同。

算法1描述了根据判别器学习一组彼此多样化的策略的过程。我们首先将 S_π 定义为要学习的固定策略集，并将 D 定义为判别器。每个回合在FightingICE中包含固定数量的“轮次”，可以视为从初始状态开始的独立片段。对于每一轮，我们采样一个策略 $\pi_r \sim S_\pi$ 来“主导”该轮。这样做是为了平衡判别器的训练数据，同时也为了有可能遇到只有通过在整个轮次中遵循同一策略才能达到的状态。在每个时间步，我们根据 π_r 使用 ϵ-贪心策略选择一个动作，其中 ϵ 在50个回合内从0.95线性退火到0.05，以确保高度的早期探索。然后，我们使用多样性奖励函数 R_d计算每个策略 π 的奖励，定义如下

$$R_d(s_t, a_t, \pi, D) = logD(s_t, a_t|\pi) - log\frac{1}{|S_\pi|} \qquad (2)$$

其中 log 为自然对数， $D(s_t, a_t|\pi)$ 是判别器 D 对状态动作对 (s_t, a_t) 来自策略 π 的预测。然后我们利用 a_t推进环境，为 DQN 智能体和判别器收集训练样本。每个轮次都与一个“随机智能体”对战，该智能体均匀随机选择动作。这样做的目的是让策略能够观察到尽可能多的独特状态。在每个回合结束时，我们根据各自的样本更新每个策略，然后根据每个状态动作对的真实来源更新判别器。实验发现，在该环境中无需使用固定回放缓冲区，这与先前研究表明大型回放缓冲区并非普遍有益的观点一致（Fedus 等人，2020）。训练判别器时，我们采用 80-10-10 的训练-验证-测试拆分，记录验证准确率以检查收敛情况。DQN 智能体和判别器均使用 Adam 优化器进行训练，分别采用均方误差和分类交叉熵作为损失函数。每次更新时，我们以 10^{-5} 的学习率和 1 的批量大小训练 5 个周期。我们的超参数选择参考了先前的研究（Halina 和 Guzdial，2021）。

虽然Brisket允许任意数量的学习策略，但在我们的评估中，我们使用此策略同时训练了三个智能体。选择这个较小的数量是为了评估是否可以在无需额外“修剪”无效策略步骤的情况下，学习到多样化且有效的策略。在经历50个回合、每个回合包含100轮次游戏后，我们收敛到了99.5%的判别器测试准确率。这在一台配备AMD Ryzen 5 3600x CPU和NVIDIA GeForce 1080 Ti GPU的单机上耗时约48小时。这表明Brisket可能对那些计算能力不足的用户（如格斗游戏开发者）具有可及性。

Algorithm 1: Diversity-based learning algorithm

Let S_π be fixed set of policies, D discriminator
for episode e **do**
  for round r **do**
    Sample $\pi_r \sim S_\pi$ to make choices during r
    for timestep t **do**
      $a_t \leftarrow \epsilon - greedy(\pi_r, s_t)$
      for policy $\pi \in S_\pi$ **do**
        // calculate reward based on D
        $r_{\pi,t} \leftarrow R_d(s_t, a_t, \pi, D)$
      end for
      Step environment $s_{t+1} \leftarrow (s_t, a_t)$
    end for
  end for
  Update each policy $\pi \in S_\pi$ with samples from e
  Update discriminator D with samples from e
end for

a given action in that state (Li 2017). In this subsection, we focus on the architecture used by each DQN agent, as well as the architecture used for the discriminator which determines the reward in the diversity-based training step.

Figure 2 depicts a visualization of the DQN architecture. The model takes in a length 143 vector representing a state s, along with a length 56 one-hot vector representing the proposed action a to evaluate. These vectors are concatenated and passed through 3 fully connected layers of 128, 64, and 32 neurons respectively. Each of these layers use rectified linear unit (ReLU) as the activation function. Our final output layer is a single node using the tanh activation function to represent a predicted reward $\hat{r} \in (-1, 1)$. The use of ReLU and the parameters of our model are informed by prior work on DQN agents for the fightingICE domain, with the notable change of predicting a reward based on a state-action pair rather than treating the model as a 56-class classification problem (Takano et al. 2018). This change was to reduce the complexity of the model's representation, allowing the model to more easily learn relationships between states and individual actions.

Brisket uses a discriminator during the diversity step in order to reward agents based on their diversity. This discriminator takes in a state-action pair as input and predicts the likelihood that this pair came from each of our agents. This likelihood is then used in the reward function for these agents, encouraging them to take more and more diverse actions from one another. The discriminator uses the same architecture as each DQN agent, with the only architectural change being at the output layer. The output layer has K nodes, where K is the number of policies to learn, and uses the softmax activation function in place of tanh. We chose to use the same architecture for both the discriminator and DQN agents, as they are modelling similar relationships between state-action pairs and their desired output.

Diversity-based Learning

Brisket is largely inspired by DIAYN, an approach for concurrently training a group of RL agents which are rewarded based on a discriminator which determines how distinct these agents are from one another (Eysenbach et al. 2018). Brisket makes the major change of using state-action input for our DQN agents and discriminator rather than just state (Eysenbach et al. 2018). This change was due to our task of behaviour generation, which differs from the exploration task for which DIAYN was originally designed.

Algorithm 1 describes the procedure for learning a set of policies that are diverse from one another according to a discriminator. We begin by defining S_π as the fixed set of policies to be learned, and D as the discriminator. Each episode contains a fixed number of “rounds” in fightingICE, which can be thought of as individual rollouts from an initial state. For each of these rounds, we sample a policy $\pi_r \sim S_\pi$ to “pilot” the given round. This is done in an attempt to balance the training data for the discriminator, as well as to allow for the possibility of encountering states which are only feasibly reachable through following the same policy for an entire round. For each timestep, we select an action according to π_r using ϵ-greedy, with ϵ being annealed from 0.95 to 0.05 linearly across 50 episodes to ensure a high amount of early exploration. We then calculate the reward for each policy π using the diversity-reward function R_d, defined as

$$R_d(s_t, a_t, \pi, D) = logD(s_t, a_t|\pi) - log\frac{1}{|S_\pi|} \qquad (2)$$

where log is the natural logarithm, and $D(s_t, a_t|\pi)$ is the discriminator D's prediction that the state-action pair (s_t, a_t) came from policy π. We then step the environment forward using a_t, collecting training samples for the DQN agents and discriminator. Each of the rounds are played against a “random agent” which selects actions uniformly at random. This was to allow for the policies to see as many unique states as possible. At the end of each episode, we update each policy based on their respective samples, then update the discriminator based on the true pilot for each state-action pair. We found experimentally that we did not need to use a fixed replay buffer in this environment, as supported by prior work which showed large replay buffers are not universally beneficial (Fedus et al. 2020). We perform an 80-10-10 train-val-test split when training the discriminator, recording the validation accuracy to check for convergence. The DQN agents and discriminator are both trained using the Adam optimizer, with MSE and categorical cross-entropy as loss functions, respectively. For each of these updates, we train for 5 epochs with a learning rate of 10^{-5} and a batch size of 1. Our selection of hyperparameters was informed by prior work (Halina and Guzdial 2021).

While Brisket allows for an arbitrary number of learned policies, for our evaluation we trained three agents concurrently using this strategy. This small number was chosen to evaluate if diverse, effective strategies could be learned without an additional step of “pruning” a number of ineffective strategies. After 50 episodes with 100 rounds of gameplay in each episode, we converged, reaching a discriminator test accuracy of 99.5%. This took approximately 48 hours to run on a single machine using an AMD Ryzen 5 3600x CPU and NVIDIA GeForce 1080 Ti GPU. This suggests Brisket may be accessible to those without substantial computing power, such as fighting game developers.

微调

在我们方法的第二步中，每个学习到的策略被单独微调到相近的难度水平。这是为了确保每个策略都不是简单到可以轻易击败的，因为我们在前一步骤中并未对获胜给予奖励。对于每个策略，我们使用之前定义的微调奖励函数 R_f 额外训练了50个回合。在此步骤中，我们将学习率降低至 10^{-6}，并使用固定的 ϵ 值为0.05，以尽量避免大幅改变策略预先学习到的行为。与多样性步骤相同，这些轮次均与随机智能体对战，以便学习尽可能通用的策略。

在每个回合之间，我们让智能体使用贪心动作选择与随机智能体进行9轮次（3场比赛，每场3轮次）的对战，并记录这些轮次的平均奖励。在经历50个回合、每个回合包含100轮次游戏后，奖励趋于平稳，我们便停止了训练。最后，我们以平均奖励为标准，选取得分最高的回合作为策略的最终版本。此举旨在确保选出每个策略中最有效的版本。

通过运行一次Brisket，我们创建了三个多样化且高效的智能体，它们利用不同的策略来赢得游戏。根据其行为，我们将这些智能体命名为“Combo”、“Rushdown”和“Sweeper”，其行为概述详见智能体描述章节。每个智能体表现最佳的版本分别来自微调回合的第17、6和20回合。我们选择仅运行一次Brisket作为初步测试，并避免因大数定律而偶然获得一组优质智能体的可能性。

评估

在本节中，我们将讨论评估标准以及用于与Brisket进行比较的人工编写的基线。我们的目标是创建一组技能水平相近、但在策略上各具特色的多维难度智能体。因此，我们需要另一组智能体作为比较对象，同时还需要能够衡量策略能力与多样性的方法。

作为基线，我们使用了三个采用手工设计的奖励函数的DQN智能体，这些奖励函数是在我们的多样性方法之前设计和训练的。这是为了避免Brisket输出的智能体对基线奖励函数的设计产生任何偏差影响。这些奖励函数的设计意图是在仍然优化最优玩法的情况下，激发出所期望的多样化行为。我们选择与DQN智能体进行比较，而不是与其他人工编写的方案（如基于规则的系统）进行比较，以使比较尽可能平衡。因此，这种比较可被视为将Brisket与我们自身创建多样化强化学习智能体的能力进行对比。根据我们期望这些智能体表现出的行为，我们将这三个智能体分别命名为“Aggressive”、“Balanced”和“Counter”。每个智能体的最佳版本分别来自第76、64和99回合。这些智能体的训练及其奖励函数在基线DQN训练小节中进行了概述，其行为在智能体描述章节中进行了总结。

我们进行了两次不同的评估，将Brisket学习到的策略与基线智能体进行比较。第一次评估是对随机状态下的多样性进行测量。为此，我们通过让两个随机智能体相互对战，收集了一万个随机状态。然后，我们让六个智能体中的每一个都贪婪地选择它们在每个状态下会采取的动作，并记录下它们的选择。接着，我们查看了每个智能体与其他智能体相比，在其中选择不同动作的状态所占的百分比。这让我们了解到，在多种多样的情境状态下，各智能体的动作在彼此之间有多大的多样性。

第二次评估采用头对头循环赛方式，每个智能体与其他智能体分别进行10场比赛。我们选择仅进行10场比赛，因为发现增加比赛场次会产生一致的结果。在这10场比赛中，每个智能体一半场次从左侧开始，另一半从右侧开始，以避免任何智能体获得不公平优势。我们记录了每个智能体在每场对决中的胜负平记录以及总记录。此次评估旨在衡量每组智能体之间的相对有效性，同时确保我们不会为了多样性而牺牲有效性——Brisket若能产生难度一致但更多样化的策略，将有力证明其能够实现多维难度。

基线DQN训练

在本小节中，我们讨论基线DQN智能体的训练过程及其各自的奖励函数。我们在训练过程中尽可能保持Brisket与基线之间参数的一致性。每个智能体均采用先前描述的相同架构。我们使用了与Brisket相同的训练时间、超参数和最终策略选择。这种一致性旨在减少除奖励函数具体变化之外影响智能体性能的其他因素。

每个智能体的奖励函数都包含使用$T(s)$（如第4.1节所定义），用于奖励智能体赢得或输掉游戏，同时引入存在性惩罚 p =1，以防止智能体停滞或陷入动作“循环”。我们通过实验发现，这一惩罚对于使每个智能体表现出合理行为是必要的。这些奖励函数如下所示。

攻击型智能体：“攻击型”智能体使用的奖励函数 R_a 旨在使智能体的行为偏向于以击中对手为首要目标。R_a 定义为

$$R_a(s,a) = T(s)\cdot 1000 + A(s)\cdot 100 - p \qquad (3)$$

其中 $A(s) = 1$ 表示智能体在 s中对对手造成了伤害，否则 $A(s) = 0$。

平衡型智能体：“平衡型”智能体使用的奖励函数 R_b 旨在让智能体在决策时同时考虑进攻和防守。R_b 定义为

$$R_b(s,a) = T(s)\cdot 1000 + A(s)\cdot 100 - B(s)\cdot 50 - p \quad (4)$$

其中 $A(s)$ 如上所述，$B(s)$ 在 s 中若智能体受到伤害则为 $=1$，否则为 $B(s) = 0$。$B(s)$ 被赋予较低权重，

Fine-tuning

In the second step of our approach, each of the learned policies are individually fine-tuned to a similar level of difficulty. This is to ensure each of the policies is not trivially easy, as we did not reward winning in the previous step. For each of these policies, we trained for an additional 50 episodes, using the fine-tuning reward function R_f defined previously. During this step, we lower the learning rate to 10^{-6} and use a fixed ϵ of 0.05 in an attempt to not drastically alter the pre-existing learned behaviour of the policies. As in our diversity step, these rounds are played against a random agent to learn the most general strategies possible.

Between every episode we had the agent play 9 rounds (3 matches of 3 rounds) against the random agent using greedy action selection, recording the average reward for these rounds. After 50 episodes with 100 rounds of gameplay in each, our reward plateaued and we stopped training. Finally, we selected the highest scoring episode in terms of average reward as the final version of the policy. This was to ensure we selected the most effective version of each policy.

By running Brisket once, we created three diverse, effective agents that utilize differing strategies to win games. We named these agents “Combo,” “Rushdown,” and “Sweeper” based on their behaviour, which is overviewed in the Agent Descriptions section. The top performing version of each agent came from fine-tuning episodes 17, 6, and 20 respectively. We chose to only run Brisket once as an initial test, and to avoid the possibility of stumbling into a good set of agents by the law of large numbers.

Evaluation

In this section we discuss the evaluation criteria and the human-authored baseline we compare Brisket against. Our goal is to create a set of multidimensionally difficult agents of similar skill levels that exhibit distinct strategies from one another. As such, we require another set of agents to compare against, as well as methods for measuring both capability and diversity of strategy.

As a baseline we used three DQN agents employing hand-authored reward functions that were designed and trained prior to our diversity-based approach. This was to avoid any chance of the output agents from Brisket biasing the design of the baseline reward functions. These reward functions were designed with the intention of eliciting desired, diverse behaviours while still optimizing for optimal play. We chose to compare against DQN agents instead of other human-authored approaches such as rules-based systems to make the comparison as balanced as possible. Therefore, this comparison can be seen as comparing Brisket against our own ability to create diverse RL agents. We named these three agents “Aggressive,” “Balanced,” and “Counter” after the behaviours we intended for them to exhibit. The top performing version of each agent came from episodes 76, 64, and 99 respectively. The training of these agents and their reward functions are outlined in the Baseline DQN Training subsection, and their behaviours are summarized in the Agent Descriptions section.

We performed two different evaluations comparing the policies learned by Brisket to the baseline agents. The first was a measurement of diversity across randomized states. To accomplish this, we collected ten thousand random states by pitting two random agents against each other. We then queried each of our six agents to greedily select the action they would take in each of these states, and recorded their choices. We then looked at the percentage of states in which each agent chose a distinct action compared to each other agent. This gives us a notion of how diverse the actions of the agents were relative to one another across a wide variety of states.

The second evaluation was a head-to-head round-robin tournament in which each agent played 10 matches against each other agent. We chose to run for only 10 matches as we found running additional games produced consistent results. In half of these matches, each agent started on the left side, and in the other half the right to avoid giving any agent an unfair advantage. We recorded the win-loss-tie records of each agent in every match-up, as well as total records. This evaluation is intended to measure the effectiveness of each set of agents in relation to one another, as well as ensuring we are not sacrificing effectiveness for diversity with Brisket. If Brisket produces more diverse strategies of a consistent difficulty, this would represent strong evidence towards it being able to achieve multidimensional difficulty.

Baseline DQN Training

In this subsection we discuss the training procedure for our baseline DQN agents, along with their individual reward functions. We kept as many parameters consistent between Brisket and the baseline as possible while training. Each agent used the same architecture described previously. We used the same training time, hyperparameters, and final policy selection as in Brisket. This consistency was intended to reduce the factors affecting the agent’s performance beyond the specific change of the reward function.

The reward function for each agent includes the use of $T(s)$ as defined in Section 4.1 to reward agents for winning or losing games, along with an existential penalty of $p = 1$ to keep agents from stalling or getting stuck in “loops” of actions. We found experimentally that this penalty was necessary to achieve reasonable behaviour from each agent. These reward functions are as follows.

Aggressive Agent: The “Aggressive” agent uses the reward function R_a designed to bias the agent’s behaviour towards hitting the opponent above all else. R_a is defined as

$$R_a(s,a) = T(s)\cdot 1000 + A(s)\cdot 100 - p \qquad (3)$$

where $A(s) = 1$ if the agent dealt damage to the opponent in s, and $A(s) = 0$ otherwise.

Balanced Agent: The “Balanced” agent uses the reward function R_b designed to make the agent take into account both offense and defense when making decisions. R_b is defined as

$$R_b(s,a) = T(s)\cdot 1000 + A(s)\cdot 100 - B(s)\cdot 50 - p \quad (4)$$

where $A(s)$ is as above, and $B(s) = 1$ if the agent was dealt damage in s, and $B(s) = 0$ otherwise. $B(s)$ is weighted

	Combo	Rushdown	Sweeper	Aggressive	Balanced	Counter	All
Combo	0%	98.59%	99.12%	99.53%	96.23%	99.75%	93.59%
Rushdown	98.59%	0%	83.87%	99.04%	97.75%	98.74%	78.73%
Sweeper	99.12%	83.87%	0%	99.94%	99.74%	99.96%	82.69%
Aggressive	99.53%	99.04%	99.94%	0%	87.89%	94.73%	83.19%
平衡	96.23%	97.75%	99.74%	87.89%	0%	78.59%	62.40%
Counter	99.75%	98.74%	99.96%	94.73%	78.59%	0%	73.75%

表1：在随机一万个状态下的多样性比较结果。“全部”列显示在某个状态下与其他所有智能体的动作均不相同的动作所占百分比。每个单元格代表两个特定智能体之间的多样性。Combo、Rushdown和Sweeper是由Brisket学习到的智能体。

	Combo	Rushdown	Sweeper	Aggressive	Balanced	Counter	总体记录
Combo	X	10 - 0 - 0	10 - 0 - 0	9 - 1 - 0	5 - 5 - 0	8 - 2 - 0	42 - 8 - 0
Rushdown	0 - 10 - 0	X	4 - 1 - 5	5 - 5 - 0	4 - 6 - 0	5 - 5 - 0	18 - 27 - 5
Sweeper	0 - 10 - 0	1 - 4 - 5	X	9 - 1 - 0	5 - 5 - 0	10 - 0 - 0	25 - 20 - 5
Aggressive	1 - 9 - 0	5 - 5 - 0	1 - 9 - 0	X	5 - 5 - 0	0 - 10 - 0	12 - 38 - 0
平衡	5 - 5 - 0	6 - 4 - 0	5 - 5 - 0	5 - 5 - 0	X	5 - 5 - 0	26 - 24 - 0
Counter	2 - 8 - 0	5 - 5 - 0	0 - 10 - 0	10 - 0 - 0	5 - 5 - 0	X	22 - 28 - 0

表2：头对头循环赛的结果，以每个行中智能体的视角按胜 - 负 - 平列出。Combo、Rushdown 和 Sweeper 是由 Brisket 学习到的智能体。

低于 $A(s)$，因为我们通过实验发现，若采用相等权重，智能体将陷入局部最优。

对抗体： “Counter” 智能体使用奖励函数 R_c，该函数旨在引导智能体在对手动作中途进行“反击”。R_c 定义为

$$R_c(s,a) = T(s) \cdot 1000 + C(s) \cdot 100 - p \qquad (5)$$

其中 $C(s) = 1$ 若智能体对处于 s 中动作中途的对手造成了伤害，否则 $C(s) = 0$。

智能体描述

由于我们无法详尽地为每个智能体列出所有策略，本部分将简要介绍各智能体在我们视角下的行为特征。我们还提供了每个智能体的游戏视频以便您获得更多背景信息[2]。

首先介绍习得的多样性技能：**Combo** 智能体以连续快速攻击为特征。该智能体根据局势使用多种招式，并习得了一套难以逃脱的连击技巧，能将对手逼入角落。**Rushdown** 智能体擅长快速接近对手并发动猛烈攻击。该智能体频繁切换高位与低位攻击，使对手难以有效格挡或防守。**Sweeper** 智能体几乎在所有情况下都倾向于使用低位扫踢。该智能体已掌握如何有效衔接这些扫踢动作，除非对手正确格挡或闪避攻击，否则将使其永久倒地。

这三个人工撰写的技能都收敛到了相似的极大值，每个策略之间只有细微差别。该极大值的达成方式为：在轮次开始时使用一个范围较广的上勾拳动作，随后持续抛射抛射物，直至智能体能量耗尽或该轮次结束。

[2]https://www.youtube.com/watch?v=GnoURvbHMLY

The **Aggressive** 智能体是这种“上勾拳”策略的最简单形式，试图在不顾自身安危的情况下对对手连续施展上勾拳。The **Balanced** 智能体偶尔会将这种基本策略与更快速的踢击混合使用，并且相比其他人类编写的智能体，它更倾向于抛射抛射物。我们推测这是由于该智能体奖励函数中蕴含的自我保护概念所致。The**Counter** 智能体也对这种基本策略进行了调整，更多地专注于上勾拳而非抛射抛射物。该智能体不将能量用于抛射抛射物，而是执行一个滑铲踢动作，该动作能够穿过大多数其他动作的下方，从而使其在对手出招过程中有效实现“反击命中”，这正是其奖励函数设计所期望的效果。

结果

在本节中，我们讨论动作多样性和头对头评估的结果。

表1比较了各智能体在一万个随机状态中所选择动作的多样性。总体而言，由Brisket学习到的智能体所采取的动作，比人类编写的基线智能体更频繁地呈现出与整个智能体群体不同的多样性。这表明基于多样性的智能体比基线智能体表现出更独特的行为。特别是，Combo智能体表现出最具多样性的行为，其选择与整个群体不同的动作的时间占93.59%。我们推测这是由于该智能体使用了高度多样的招式，如第6节所述。相比之下，人类编写的智能体采取的多样性动作较少，我们推测这是由于其奖励函数虽有差异，但各智能体仍收敛到相似的局部最大值所致。这可能暗示了在同时优化期望的多样化行为和最优玩法方面存在弱点，而Brisket通过其两步流程规避了这一问题。值

	Combo	Rushdown	Sweeper	Aggressive	Balanced	Counter	All
Combo	0%	98.59%	99.12%	99.53%	96.23%	99.75%	93.59%
Rushdown	98.59%	0%	83.87%	99.04%	97.75%	98.74%	78.73%
Sweeper	99.12%	83.87%	0%	99.94%	99.74%	99.96%	82.69%
Aggressive	99.53%	99.04%	99.94%	0%	87.89%	94.73%	83.19%
Balance	96.23%	97.75%	99.74%	87.89%	0%	78.59%	62.40%
Counter	99.75%	98.74%	99.96%	94.73%	78.59%	0%	73.75%

Table 1: The results of the diversity comparison across ten thousand random states. The “All” columns depicts the percentage of actions taken in a state that were diverse from all other agents. Each cell represents the diversity between two specific agents. Combo, Rushdown, and Sweeper are the agents learned by Brisket.

	Combo	Rushdown	Sweeper	Aggressive	Balanced	Counter	Overall Record
Combo	X	10 - 0 - 0	10 - 0 - 0	9 - 1 - 0	5 - 5 - 0	8 - 2 - 0	42 - 8 - 0
Rushdown	0 - 10 - 0	X	4 - 1 - 5	5 - 5 - 0	4 - 6 - 0	5 - 5 - 0	18 - 27 - 5
Sweeper	0 - 10 - 0	1 - 4 - 5	X	9 - 1 - 0	5 - 5 - 0	10 - 0 - 0	25 - 20 - 5
Aggressive	1 - 9 - 0	5 - 5 - 0	1 - 9 - 0	X	5 - 5 - 0	0 - 10 - 0	12 - 38 - 0
Balance	5 - 5 - 0	6 - 4 - 0	5 - 5 - 0	5 - 5 - 0	X	5 - 5 - 0	26 - 24 - 0
Counter	2 - 8 - 0	5 - 5 - 0	0 - 10 - 0	10 - 0 - 0	5 - 5 - 0	X	22 - 28 - 0

Table 2: The results of the head-to-head round-robin tournament, listed by win - loss - tie from the perspective of the agent in each row. Combo, Rushdown, and Sweeper are the agents learned by Brisket.

lower than $A(s)$, as we found experimentally that the agent would get trapped in local maxima with equal weighting.

Counter Agent: The “Counter” agent uses the reward function R_c designed to bias the agent into “counter-attacking” the opponent while they are in the middle of a move. R_c is defined as

$$R_c(s,a) = T(s) \cdot 1000 + C(s) \cdot 100 - p \qquad (5)$$

where $C(s) = 1$ if the agent dealt damage to an opponent that was mid-move in s, and $C(s) = 0$ otherwise.

Agent Descriptions

Because we cannot realistically give all the policies for each agent, in this section we briefly describe each agent’s behaviour from our perspective. We also provide a video of the gameplay of each agent for additional context[2].

Beginning with the learned diversity skills, **Combo** is an agent characterized by its chaining of fast moves together. The agent uses a wide variety of moves depending on the situation, and has learned a difficult-to-escape combo that traps the opponent in the corner. **Rushdown** is an agent which quickly approaches the opponent to relentlessly attack. The agent frequently switches between high and low attacks, making it difficult for an opponent to effectively block or defend themself. **Sweeper** is an agent that chooses to use a low sweeping kick in almost every circumstance. The agent has learned to effectively chain these sweeps together, keeping the enemy permanently knocked down unless they correctly block or evade the attack.

The three human-authored skills all converged to similar maxima, with small distinctions between each policy. This maxima involves using a wide-reaching uppercut move during the beginning of a round, followed by throwing projectiles until the agent is out of energy or the round is finished.

[2]https://www.youtube.com/watch?v=GnoURvbHMLY

The **Aggressive** agent is the simplest form of this “uppercut” strategy, attempting to chain uppercuts against the opponent without caring for self-preservation. The **Balanced** agent occasionally mixes up this base strategy with faster kicks, and has a higher preference toward throwing projectiles than the other human-authored agents. We hypothesize that this is due to the notion of self-preservation in this agent’s reward function. The **Counter** agent also tweaks this base strategy by focusing more on uppercutting than throwing projectiles. Instead of using energy on throwing projectiles, this agent performs a sliding kick move that goes under most other moves, allowing the agent to effectively “counter hit” the opponent mid-move as desired by the design of its reward function.

Results

In this section we discuss the results of the action diversity and head-to-head evaluations.

Table 1 compares the diversity between each agent’s chosen actions across ten thousand random states. Overall, the agents learned by Brisket took actions that were diverse from the entire population of agents more often than the human-authored baseline agents. This suggests the diversity-based agents are exhibiting more distinct behaviours than the baseline agents. In particular, the Combo agent exhibited the most diverse behaviour, selecting actions that were diverse from the entire population 93.59% of the time. We hypothesize this is due to the high variety of moves used by the agent, as described in Section 6. By comparison, the human-authored agents took less diverse actions, which we hypothesize is a result of the agents converging to similar local maxima despite the differences in their reward functions. This may suggest a weakness in simultaneously optimizing towards desired diverse behaviours and optimal play, which Brisket subverts through its two-step process. No-

得注意的是，尽管攻击型智能体在总体多样性上优于突击型智能体和扫荡型智能体，但这主要是因为其与基于多样性的智能体之间存在差异。从我们的角度来看，Brisket的智能体彼此之间的区别明显比人类编写的智能体更显著，这凸显了在没有人类评估的情况下测量人类主观多样性的难度。

表2展示了六种DQN智能体之间直接对抗赛的结果。总体而言，Brisket在锦标赛中的表现优于人工编写的基线，两组智能体分别累计取得了85胜-55负-10平与60胜-90负-0平的战绩。这一表现令人鼓舞，表明我们并未为了追求多样化的行为而牺牲有效性。两组智能体显示出相似的内部平衡性，每组各有两名得分相等的智能体，以及一名离群值。值得注意的是，尽管Combo和Sweeper在多样性实验中几乎完全不同，但它们与人类智能体对抗时的表现却非常相似。这朝着我们利用Brisket实现多维难度的目标迈出了令人充满希望的一步。

讨论

在本节中，我们将反思实验结果，探讨评估及系统设计的局限性，并讨论未来工作的潜在方向。

局限性

我们工作中最显著的局限性在于缺乏人类评估。尽管由Brisket生成的智能体在我们的评估指标上优于基线，但缺乏人类评估使得难以对这些智能体的质量或有效性得出强有力的结论。特别是，我们的多样性评估结果突显了衡量像多样性这样具有主观性和定性特征的指标是多么困难。同样，人类评估将有助于评估所学智能体的鲁棒性，因为当前策略可能存在脆弱性或无法有效对抗人类对手的可能性。尽管人类评估超出了本文的范围，但我们认为这是指导Brisket未来发展的明确下一步。

此外，我们的评估与结果也存在若干局限性。两项评估的样本量都相对较小，这可能带来潜在的偏差来源。然而，在多样性评估和直接对抗赛这两种情况下，我们发现使用更多样本所得结果几乎完全相同。同样，在本文中，由于计算资源的限制，我们仅利用Brisket学习了少量智能体。尽管Brisket在处理更多学习策略时可能更有效，但我们认为我们的结果表明该系统对于没有强大计算能力的开发者是有用的，这是一个积极信号。

未来工作

虽然在本文中，我们提出了一种用于创建具有多维难度的智能体的潜在方法，但未来仍有其他可能的探索途径。其中一条途径可以是，在训练中使用基于状态的多样性指标，根据每个智能体所遇到的游戏状态的多样性来给予奖励。这可能会比我们目前基于动作的多样性指标产生更多样的行为，并且两者可以协同使用，以创建能够通过彼此不同的动作将比赛引导至不同状态的智能体。此外，质量-多样性方法也可能是该问题的另一种解决方案，据我们所知，该方法尚未应用于行为生成领域（Gravina 等人，2019）。

结论

本文提出了Brisket，一种基于多样性的深度强化学习方法，旨在生成多维难度性的格斗游戏智能体。我们在FightingICE中实现了Brisket，生成了三个智能体，与使用人工编写的奖励函数的DQN智能体进行对比。我们发现Brisket在多样性和有效性两方面均优于这个人工编写的基线。我们希望Brisket能成为游戏中实现多维难度的初步探索，并激发更多相关工作对此问题进行深入研究。

致谢

我们感谢阿尔伯塔省机器智能研究所（Amii）的支持。

参考文献

Barros, G. A.; Liapis, A.; 和 Togelius, J. 2016. 数据游戏：基于开放数据的冒险程序化生成。Butler, E.; Siu, K.; 和 Zook, A. 2017. 程序合成作为一种生成方法。*FDG*, 6:1–6:10。Chen, T.; Richoux, F.; Torres, J. M.; 和 Inoue, K. 2021. 应用于FightingICE平台的可解释基于效用模型。*2021 IEEE*游戏会议（*CoG*）, 1–8。IEEE。Cook, M.; Colton, S.; 和 Gow, J. 2016. AngeliNA电子游戏设计系统——第二部分。*IEEE*计算智能与游戏中的人工智能汇刊, 9(3): 254–266。Delarosa, O.; Dong, H.; Ruan, M.; Khalifa, A.; 和 Togelius, J. 2021. 使用RL画笔的混合主动式关卡设计。国际音乐、声音、艺术与设计计算智能会议（*EvoStar*部分）, 412–426。斯普林格。Dhami, N. 2021. 格斗游戏中的玩家表达。*Medium*。Eysenbach, B.; Gupta, A.; Ibarz, J.; 和 Levine, S. 2018. 多样性即是一切：无需奖励函数学习技能。*arXiv*预印本 *arXiv:1802.06070*。Fedus, W.; Ramachandran, P.; Agarwal, R.; Bengio, Y.; Larochelle, H.; Rowland, M.; 和 Dabney, W. 2020. 重新审视经验回放的基础。国际机器学习大会, 3061–3071。PMLR。

tably, while the Aggressive agent outperforms the Rushdown and Sweeper agents in total diversity, this is primarily due to the differences between it and the diversity-based agents. From our perspective, Brisket's agents are noticeably more distinct from one another than the human-authored agents, which highlights the difficulty of measuring human-subjective diversity without human evaluation.

Table 2 depicts the results of the head-to-head tournament between the six DQN agents. Overall, Brisket outperformed the human-authored baseline in the tournament, with cumulative records of 85 wins - 55 losses - 10 ties and 60 wins - 90 losses - 0 ties respectively across both groups of agents. This performance is promising, as it suggests that we did not sacrifice effectiveness for the sake of diverse behaviour. The groups of agents showed a similar degree of internal balance, with each having two equally scoring agents, and one outlier. Notably, Combo and Sweeper performed very similarly versus the human agents despite being almost completely distinct from one another in the diversity experiment. This is a promising sign toward our goal of attaining multidimensional difficulty with Brisket.

Discussion

In this section we reflect upon our results, addressing the limitations of our evaluation and system design and discussing potential avenues for future work.

Limitations

The most notable limitation of our work is the lack of human evaluation. While the agents generated by Brisket outperformed the baseline on our evaluation metrics, the lack of human evaluation makes it difficult to draw strong conclusions about the quality or effectiveness of these agents. In particular, the results of our diversity evaluation highlight the difficulty in measuring something as qualitative and subjective as diversity. As well, human evaluation would be helpful for evaluating the robustness of the learned agents, as there is a possibility the current strategies may be brittle or ineffective against human opponents. While human evaluation is outside the scope of this paper, we feel it is a clear next step to guide future development of Brisket.

As well, there are a number of limitations concerning our evaluation & result. Both of the evaluations have relatively small sample sizes, which could present a potential source of bias. However, in both the case of the diversity evaluation and head-to-head tournament, we found that using additional samples yielded almost identical results. As well, in this paper we only utilized Brisket to learn a small number of agents due to computational constraints. While Brisket may potentially be more effective with a larger number of learned policies, we interpret our results as a positive sign that the system may be useful to developers without substantial computing power.

Future Work

While in this paper we present one potential method for creating agents that exhibit multidimensional difficulty, there are other possible avenues for future exploration. One such avenue could be the use of a state-based diversity metric in training which rewards agents based on the diversity of game states each agents encounters. This may yield more diverse behaviour than our current action-based diversity metric, and the two could potentially be used in tandem to create agents which drive a match towards distinct states using disparate actions from one another. As well, quality-diversity methods could represent another solution to this problem, and have yet to be applied in the domain of behaviour generation to our knowledge (Gravina et al. 2019).

Conclusions

In this paper we presented Brisket, a diversity-based deep RL approach with the goal of generating multidimensionally difficult fighting game agents. We implemented Brisket in FightingICE, generating three agents to compare against a set of DQN agents using human-authored reward functions. We found Brisket outperformed this human-authored baseline in both diversity and effectiveness. We hope Brisket can represent a preliminary step towards multidimensional difficulty in games, and inspire other work to explore the problem further.

Acknowledgements

We acknowledge the support of the Alberta Machine Intelligence Institute (Amii).

References

Barros, G. A.; Liapis, A.; and Togelius, J. 2016. Playing with data: Procedural generation of adventures from open data.

Butler, E.; Siu, K.; and Zook, A. 2017. Program synthesis as a generative method. In *FDG*, 6:1–6:10.

Chen, T.; Richoux, F.; Torres, J. M.; and Inoue, K. 2021. Interpretable Utility-based Models Applied to the FightingICE Platform. In *2021 IEEE Conference on Games (CoG)*, 1–8. IEEE.

Cook, M.; Colton, S.; and Gow, J. 2016. The angelina videogame design system—part ii. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(3): 254–266.

Delarosa, O.; Dong, H.; Ruan, M.; Khalifa, A.; and Togelius, J. 2021. Mixed-initiative level design with rl brush. In *International Conference on Computational Intelligence in Music, Sound, Art and Design (Part of EvoStar)*, 412–426. Springer.

Dhami, N. 2021. Player expression in fighting games. *Medium*.

Eysenbach, B.; Gupta, A.; Ibarz, J.; and Levine, S. 2018. Diversity is all you need: Learning skills without a reward function. *arXiv preprint arXiv:1802.06070*.

Fedus, W.; Ramachandran, P.; Agarwal, R.; Bengio, Y.; Larochelle, H.; Rowland, M.; and Dabney, W. 2020. Revisiting fundamentals of experience replay. In *International Conference on Machine Learning*, 3061–3071. PMLR.

Gravina, D.; Khalifa, A.; Liapis, A.; Togelius, J.; 和 Yannakakis, G. N. 2019. 通过质量多样性进行程序化内容生成。*2019 IEEE*游戏会议（*CoG*）, 1–8。IEEE。Gregor, K.; Rezende, D. J.; 和 Wierstra, D. 2016. 变分内在控制。*arXiv*预印本 *arXiv:1611.07507*。Guzdial, M.; 和 Riedl, M. 2018. 通过概念扩展的自动化游戏设计。*AAAI*人工智能与互动数字娱乐会议论文集, 14(1): 31–37。Halina, E.; 和 Guzdial, M. 2021. Taikonation：针对节奏动作游戏的模式聚焦谱面生成。第*16*届国际数字游戏基础会议（*FDG*）*2021*, 1–10。Hendrikx, M.; Meijer, S.; Van Der Velden, J.; 和 Iosup, A. 2013. 游戏程序化内容生成：综述。*ACM*多媒体计算、通信与应用汇刊（*TOMM*）, 9(1): 1–22。Ishihara, M.; Miyazaki, T.; Chu, C. Y.; Harada, T.; 和 Thawonmas, R. 2016. 在格斗游戏AI中应用并改进蒙特卡洛树搜索。第*13*届计算机娱乐技术国际进展会议论文集, 1–6。Ishii, R.; Ito, S.; Ishihara, M.; Harada, T.; 和 Thawonmas, R. 2018. 具有角色性格的格斗游戏AI的蒙特卡洛树搜索实现。*2018 IEEE*计算智能与游戏会议（*CIG*）, 1–8。IEEE。Khalifa, A.; Bontrager, P.; Earle, S.; 和 Togelius, J. 2020. PCGRL：通过强化学习进行程序化内容生成。*AAAI*人工智能与互动数字娱乐会议论文集, 第16卷, 95–101。Kim, D.-W.; Park, S.; 和 Yang, S.-i. 2020. 使用深度强化学习与自我对弈掌握格斗游戏。*2020 IEEE*游戏会议（*CoG*）, 576–583。IEEE。Kim, M.-J.; 和 Ahn, C. W. 2018. 使用遗传算法和蒙特卡洛树搜索的混合格斗游戏人工智能。遗传与进化计算会议伴生论文集, 129–130。Li, Y. 2017. 深度强化学习：概述。*arXiv*预印本 *arXiv:1701.07274*。Lu, F.; Yamamoto, K.; Nomura, L. H.; Mizuno, S.; Lee, Y.; 和 Thawonmas, R. 2013. 格斗游戏人工智能竞赛平台。*2013 IEEE*第二届全球消费电子大会（*GCCE*）, 320–323。IEEE。Majchrzak, K.; Quadflieg, J.; 和 Rudolph, G. 2015. 用于格斗游戏AI的高级动态脚本编写。国际娱乐计算会议, 86–99。斯普林格。Mitsis, K.; Kalafatis, E.; Zarkogianni, K.; Mourkousis, G.; 和 Nikita, K. S. 2020. 基于遗传算法的程序化内容生成在阻塞性睡眠呼吸暂停严肃游戏中的应用。*2020 IEEE*游戏会议（*CoG*）, 694–697。IEEE。

Oh, I.; Rho, S.; Moon, S.; Son, S.; Lee, H.; 和 Chung, J. 2021. 使用深度强化学习为实时格斗游戏打造专业级人工智能。*IEEE* 游戏汇刊. Sato, N.; Temsiririrkkul, S.; Sone, S.; 和 Ikeda, K. 2015. 通过切换多个基于规则的控制器实现自适应格斗游戏电脑玩家。见 *2015*第三届应用计算与信息技术国际会议/第二届计算科学与智能国际会议， 52–59. IEEE. Such, F. P.; Madhavan, V.; Conti, E.; Lehman, J.; Stanley, K. O.; 和 Clune, J. 2017. 深度神经进化：遗传算法是训练用于强化学习的深度神经网络的竞争力替代方案。 *arXiv*预印本 *arXiv:1712.06567*. Takano, Y.; Ouyang, W.; Ito, S.; Harada, T.; 和 Thawonmas, R. 2018. 将混合奖励架构应用于格斗游戏人工智能。 见 *2018*年*IEEE*计算智能与游戏会议（*CIG*）， 1–4. IEEE. Yuda, K.; Mozgovoy, M.; 和 Danielewicz-Betz, A. 2019. 使用Gamygdala创建情感化格斗游戏人工智能系统。 见*2019*年*IEEE*游戏会议（*CoG*）， 1–4. IEEE. Zakaria, Y.; Fayek, M.; 和 Hadhoud, M. 2022. 基于深度学习的推箱子程序化关卡生成：一项实验研究。 *IEEE* 游戏汇刊。

Gravina, D.; Khalifa, A.; Liapis, A.; Togelius, J.; and Yannakakis, G. N. 2019. Procedural content generation through quality diversity. In *2019 IEEE Conference on Games (CoG)*, 1–8. IEEE.

Gregor, K.; Rezende, D. J.; and Wierstra, D. 2016. Variational intrinsic control. *arXiv preprint arXiv:1611.07507*.

Guzdial, M.; and Riedl, M. 2018. Automated Game Design via Conceptual Expansion. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 14(1): 31–37.

Halina, E.; and Guzdial, M. 2021. Taikonation: Patterning-focused chart generation for rhythm action games. In *The 16th International Conference on the Foundations of Digital Games (FDG) 2021*, 1–10.

Hendrikx, M.; Meijer, S.; Van Der Velden, J.; and Iosup, A. 2013. Procedural content generation for games: A survey. *ACM Transactions on Multimedia Computing, Communications, and Applications (TOMM)*, 9(1): 1–22.

Ishihara, M.; Miyazaki, T.; Chu, C. Y.; Harada, T.; and Thawonmas, R. 2016. Applying and improving Monte-Carlo Tree Search in a fighting game AI. In *Proceedings of the 13th international conference on advances in computer entertainment technology*, 1–6.

Ishii, R.; Ito, S.; Ishihara, M.; Harada, T.; and Thawonmas, R. 2018. Monte-carlo tree search implementation of fighting game ais having personas. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, 1–8. IEEE.

Khalifa, A.; Bontrager, P.; Earle, S.; and Togelius, J. 2020. Pcgrl: Procedural content generation via reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 16, 95–101.

Kim, D.-W.; Park, S.; and Yang, S.-i. 2020. Mastering Fighting Game Using Deep Reinforcement Learning With Self-play. In *2020 IEEE Conference on Games (CoG)*, 576–583. IEEE.

Kim, M.-J.; and Ahn, C. W. 2018. Hybrid fighting game AI using a genetic algorithm and Monte Carlo tree search. In *Proceedings of the genetic and evolutionary computation conference companion*, 129–130.

Li, Y. 2017. Deep reinforcement learning: An overview. *arXiv preprint arXiv:1701.07274*.

Lu, F.; Yamamoto, K.; Nomura, L. H.; Mizuno, S.; Lee, Y.; and Thawonmas, R. 2013. Fighting game artificial intelligence competition platform. In *2013 IEEE 2nd Global Conference on Consumer Electronics (GCCE)*, 320–323. IEEE.

Majchrzak, K.; Quadflieg, J.; and Rudolph, G. 2015. Advanced dynamic scripting for fighting game AI. In *International Conference on Entertainment Computing*, 86–99. Springer.

Mitsis, K.; Kalafatis, E.; Zarkogianni, K.; Mourkousis, G.; and Nikita, K. S. 2020. Procedural content generation based on a genetic algorithm in a serious game for obstructive sleep apnea. In *2020 IEEE Conference on Games (CoG)*, 694–697. IEEE.

Oh, I.; Rho, S.; Moon, S.; Son, S.; Lee, H.; and Chung, J. 2021. Creating pro-level AI for a real-time fighting game using deep reinforcement learning. *IEEE Transactions on Games*.

Sato, N.; Temsiririrkkul, S.; Sone, S.; and Ikeda, K. 2015. Adaptive fighting game computer player by switching multiple rule-based controllers. In *2015 3rd international conference on applied computing and information technology/2nd international conference on computational science and intelligence*, 52–59. IEEE.

Such, F. P.; Madhavan, V.; Conti, E.; Lehman, J.; Stanley, K. O.; and Clune, J. 2017. Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. *arXiv preprint arXiv:1712.06567*.

Takano, Y.; Ouyang, W.; Ito, S.; Harada, T.; and Thawonmas, R. 2018. Applying hybrid reward architecture to a fighting game AI. In *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, 1–4. IEEE.

Yuda, K.; Mozgovoy, M.; and Danielewicz-Betz, A. 2019. Creating an affective fighting game AI system with gamygdala. In *2019 IEEE Conference on Games (CoG)*, 1–4. IEEE.

Zakaria, Y.; Fayek, M.; and Hadhoud, M. 2022. Procedural Level Generation for Sokoban via Deep Learning: An Experimental Study. *IEEE Transactions on Games*.